Comparative Analysis of Random Walkers: Linear Congruent Method vs. Random Library

SM Anis(Nahian)

January 2025

1 Algorithm

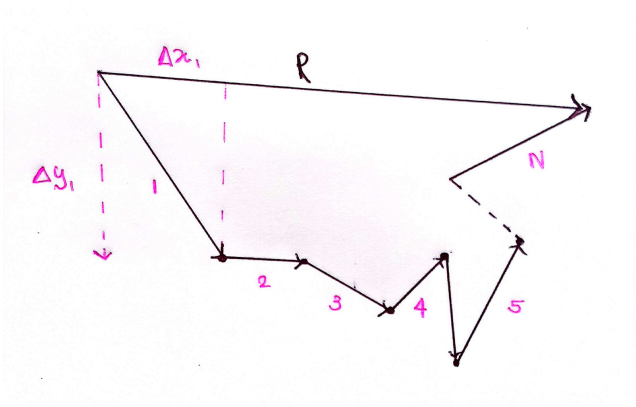

Figure 1: A schematic of N steps in a random walk simulation that end up a distance R from the origin

We will present the simplest approach for a 2D walk and end up with a model for normal diffusion. In our random walk simulation (Figure 1), an artificial walker takes sequential steps with the direction of each step independent of the direction of the previous step. In this model, we start at the origin and take N steps in the xy plane of lengths

$$(\Delta x_1, \Delta y_1) \quad (\Delta x_1, \Delta y_1) \quad (\Delta x_1, \Delta y_1) \quad (\Delta x_1, \Delta y_1) \tag{1}$$

The radial distance R from the starting point after N steps is

$$R^2 = (\Delta x_1 + \Delta x_2 + \Delta x_3 + \Delta x_4 + \cdots + \Delta x_N)^2 + (\Delta y_1 + \Delta y_2 + \Delta y_3 + \Delta y_4 + .. + \Delta y_N)^2 \tag{2}$$

$$= \Delta x_1^2 + \Delta x_2^2 + \Delta x_3^2 + \Delta x_4^2 + ... + \Delta x_N^2 + 2\Delta x_1\Delta x_2 + 2\Delta x_2\Delta x_3 + 2\Delta x_3\Delta x_4 + (x \to y) \tag{3}$$

As the walk is random, if we take the average of a large number of random steps, then all cross terms in (3) will vanish. Then we will be left with

$$R_{\text{rms}}^2 \approx \left\langle \Delta x_1^2 + \Delta x_2^2 + \Delta x_3^2 + \Delta x_4^2 + ... + \Delta x_N^2 + \Delta y_1^2 + \Delta y_2^2 + \Delta y_3^2 + \Delta y_4^2 + ... + \Delta y_N^2 \right\rangle \tag{4}$$

$$= \left\langle \Delta x_1^2 + \Delta y_1^2 \right\rangle + \left\langle \Delta x_2^2 + \Delta y_2^2 \right\rangle \ldots \tag{5}$$

$$= N \left\langle r^2 \right\rangle = N r_{\text{rms}}^2 \tag{6}$$

$$R_{\text{rms}} \approx \sqrt{N} r_{\text{rms}} \tag{7}$$

where r_{rms} is the root-mean step size .
To summarize, if the walk is random, then we expect that after a large number of steps, the average vector distance from the origin will vanish:

$\langle R\rangle = \langle x\rangle\, i + \langle y\rangle\, j \approx 0$

It means that the vector end points are distributed uniformly in all quadrants, and so the displacement vector averages to zero, but the average length of the vector does not. If we use a good random number generator, then our simulation should be in good agreement with equation(7).

1.1 Implementation: Random Walk

When we simulate the random walk, we should expect to obtain (7) only as the average displacement averaged over many trials, not necessarily as the answer for each trial. First, we use the linear congruential method as a random number generator to simulate the random walker, and then we use random.random() function in Python's random module, which generates random numbers using the Mersenne Twister algorithm, which is a pseudorandom number generator (PRNG).

We start at the origin and take 2D random walk with computer.

1. First, we generate random numbers for the x-axis using random number generators. For the linear congruential method, values for the y-axis we generated the random numbers in the manner $y_i = x_{2i+1}$. On the other hand, for the random library, we generate the random numbers individually for the x and y axes using the random() function and for different seeds.
2.We calculate the $\Delta x's$ and $\Delta y's$ for N=1000 steps.
3.We determine mean square distance R^2 for each trial and then take average of R^2 for all k trials:

$\left\langle R^2(N)\right\rangle = \frac{1}{K}\sum_{k=1}^{K} R^2_{(\text{k})}$

4.Both $\sqrt{R^2}$ and R^2_{rms} are plotted with respect to $\sqrt{N}$. Values of N should start with small numbers where $R \approx \sqrt{N}$ is not expected to be accurate and end at a quite large value, where two or three places of accuracy should be expected on average.

2 Code Implementation

2.1 Codes in Python for Random Walker Using Linear Congruent Method

```
from matplotlib import pyplot as plt
import numpy as np

n=int(input('Enter the number of trials (4 is recommanded)'))
ri=[]
a=[]
c=[]
M=[]

A=float(input('input the A')) #Upper limit of sequence
B=float(input('input the B'))#Lower limit of the sequence
k=int(input('Enter the number of Steps N'))
kx=(k*2)+3 #total number of iteration for generation ofx's


for z in range(0,n,1):
 rii=float(input('input the seed number')) #this is seed number
 ai=float(input('input the a')) #constant
 ci=float(input('input the c'))#constant
 Mi=float(input('input the M  (This is the period of sequence. It must be equal to at lest N
    ) '))# This defines the period of the sequence
 ri.append(rii)
 a.append(ai)
 c.append(ci)
 M.append(Mi)


print('The seeds are=',ri)
```

```
print("The a's are",a)
print("The c's are",c)
print("The M's are",M)




def f(j):  #It will return Rms^2 values for a jth input values
 delta_x=[]
 delta_y=[]
 a1=a[j]
 r1=ri[j]
 c1=c[j]
 M1=((M[j])*2)+3
 for i in range (0,(kx),1): #Loop------- (1)
    ri1=((((a1)*(r1))+(c1))%M1) #Bug
    xi=(A+((B-A)*(ri1/M1)))
    delta_x.append(xi)
    r1=ri1
 for b in range (0,k,1): #Loop------- (2)
   delta_y.append(delta_x[(2*b)+1])

#As the loop 1 will generate kx numbers for x axis.Now  we
#want the numbers of y axis to be y_i=x_2i +1 . So we can take only less than half the

 delta_x= delta_x[:k] #only first k numbers will remain in the list
 delta_y= delta_y[:k] ##only first k numbers will remain in the list



 R2l=[]
 Rx2=0
 Ry2=0



 for v in range(0,k,1): # Loop-----------(5)#s
    Rx2= Rx2+((delta_x[v])**2)
    Ry2=Ry2+((delta_y[v])**2)


    R2l.append(((Rx2))+((Ry2)))

 R2_rms=[0]
 N=np.arange(0,k,1)

 for g in range(0,k,1): #------------(6)
      R2_rms.append((R2l[g]))
 R2=(R2_rms)









 return (R2)







for q in range (0,n,1):  #--------(7)
 N=np.arange(0,(k+1),1)
 root_N=np.sqrt(N)
 plt.plot(root_N,(np.sqrt(f(q))), label='R of Trial'+str(q+1))
```

```
plt.legend()

#=====================================================================
#Calculaton of RMS
#=====================================================================

R_sq=[[] for _ in range(n)] #---------(8)

for o in range (0,n,1):  #---------(9)
    R_sq[o]=(f(o)) #bug



Rms_sq_avg=[]
for h in range(0,(k+1),1):  #---------(10)
 sumx=0
 for c in range(0,n,1):
  sumx=sumx+(R_sq[c][h])
 Rms_sq_avg.append(sumx/n)

Rms=np.sqrt(Rms_sq_avg)









RootN=np.sqrt(np.arange(0,(k+1),1))

plt.plot(RootN,Rms,label=' $R_{rms}$')
plt.xlim=(0,len(RootN))
plt.xlabel('$\sqrt{N}$')

plt.legend()
plt.grid()
plt.savefig("Random walker LCM.png", dpi=600)
plt.show()
```

2.2 Codes in Python using the random Module Which Generates Random Numbers Using The Mersenne Twister algorithm

```
from matplotlib import pyplot as plt
import numpy as np
import random
n=int(input('Enter the number of trials (4 is recommanded)'))
rix=[]
riy=[]
for z in range(0,n,1):
 rii=float(input('input the seed number for x')) #this is seed number
 rii2=float(input('input the seed number for y')) #this is seed number
 rix.append(rii)
 riy.append(rii2)



A=float(input('input the A')) #Upper limit of sequence
B=float(input('input the B'))#Lower limit of the sequence
k=int(input('Enter the number of steps '))

print('The seeds for x are=',rix)
print('The seeds for y are=',riy)

def f(j):  #It will return Rms^2 values for a jth input values
 del_x=[]
 del_y=[]



 random.seed(rix[j])
 for i in range (0,(k),1): #Loop------- (1)

    xi=(A+((B-A)*random.random()))
```

```
    del_x.append(xi)
 random.seed(riy[j])
 for b in range (0,(k),1):   #Loop------- (2)

     yi=(A+((B-A)*random.random()))
     del_y.append(yi)









 R2l=[]
 Rx2=0
 Ry2=0
 for v in range(0,(k),1): # Loop-----------(5)
    Rx2= Rx2+((del_x[v])**2)
    Ry2=Ry2+((del_y[v])**2)

    R2l.append(((Rx2))+((Ry2)))

 R2_rms=[0]
 N=np.arange(0,k,1)
 for g in range(0,(k),1):
      R2_rms.append((R2l[g]))
 rms2=(R2_rms)


 return (rms2)

for q in range (0,n,1):
    N=np.arange(0,k+1,1)
    plt.plot((np.sqrt(N)),np.sqrt((f(q))),label='R of Trial'+str(q+1))
plt.legend()




#Average

Rms_sq=[[] for _ in range(n)]

for o in range (0,n,1):
    Rms_sq[o]=(f(o)) #bug



Rms_sq_avg=[]
for h in range(0,(k+1),1):
 sumx=0
 for c in range(0,n,1):
  sumx=sumx+(Rms_sq[c][h])
 Rms_sq_avg.append(sumx/n)

Rms_avg=np.sqrt(Rms_sq_avg)




N=np.arange(0,(k+1),1)

RootN=np.sqrt(N)

plt.plot(RootN,Rms_avg,label='$R_{rms}$ ')
plt.xlim=(0,len(RootN))
plt.xlabel('$\sqrt{N}$')
plt.grid()
```

```
plt.legend()

plt.savefig("Random walker random().png", dpi=600)
plt.show()
```

3 Results

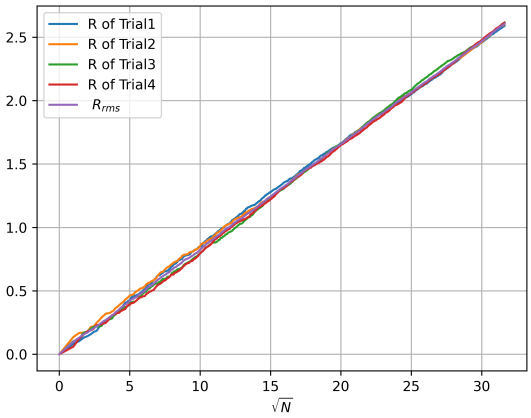

Figure 2: Using Linear Congruent Method

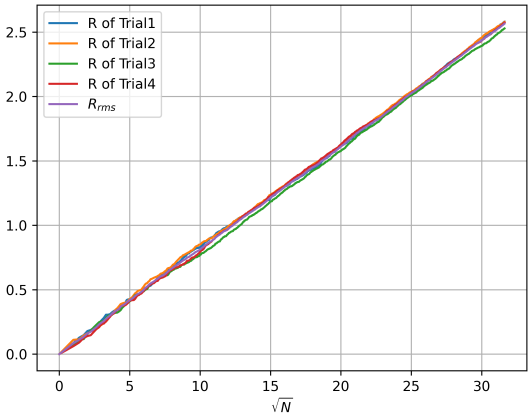

Figure 3: Using random library

For the Linear Congruent Method, the following inputs are given,
The seeds are [15.0, 105.0, 598.0, 37895.0]
The a's are [20.0, 225.0, 999.0, 6666.0]
The c's are [25.0, 365.0, 5555.0, 85214.0]
The M's are [1000.0, 1000.0, 1000.0, 1000.0]

For a random library, the following inputs are given,
The seeds for x are= [150.0, 854.0, 2999.0, 22222.0]
The seeds for y are [222.0, 3333.0, 8516.0, 56999.0]
For both methods, the upper limit and the lower limit of Δx and Δy were -0.1 and 0.1, respectively.

4 Discussion

Total distance and root-mean-square R_{rms} distance with respect to the root of steps $\sqrt{N}$ are plotted for both random walkers developed using the linear congruential method and python random library. The total number of steps was N=1000. From the algorithm, it is clear that the better the generator, is good, the more the R_{rms} vs $\sqrt{N}$ plot will follow equation (7). For the Linear Congruent Method(Figure 2), for $\sqrt{N}$ from 0 to 15, R of all trials deviates from equation (7), but after $\sqrt{N}$=15 the curve is in well agreement with equation (7). For the walker developed using the random library, R's of all trials (Figure 3) are in good agreement from the very beginning, but from $\sqrt{N}$=7.5 to 15 there is a small deviation from equation (7) in comparison with that of the linear congruential method. And after $\sqrt{N}$=15 it is showing further agreement with equation (7). Therefore, although the linear congruent method is an excellent random number generator, the random() function of the random library (This library uses the Mersenne Twister algorithm) is a much better random number generator.

5 References

1. Computational Physics: Problem Solving with Python by Rubin H. Landau (Author), Manuel J. Páez (Author), Cristian C. Bordeianu (Author)
2.https://docs.python.org/3/library/random.html